



Arquivos e Persistência

Aula 13 · Bloco 4 — Java Moderno

Fazer o sistema lembrar dos dados entre execuções

Nesta aula

1. Por que os dados **somem**
2. `Path` e `Files`
3. **Escrever e ler**
4. De **objeto** para linha, e de volta — o CSV
5. Salvar e carregar o **sistema inteiro**

Por que os dados somem

Tudo o que os seus programas fizeram até agora vive na **memória RAM**: rápida, cara e **volátil**.

Ao encerrar o processo, o sistema operacional recupera a memória — e seus objetos deixam de existir.

Para sobreviver, os dados precisam ir para **armazenamento persistente**.



O ciclo da persistência



Path e Files

A API moderna (`java.nio.file`) resolve quase tudo com duas classes:

```
Path arquivo = Path.of("dados", "acervo.csv");    // dados/acervo.csv

Files.exists(arquivo);                            // true/false
arquivo.toAbsolutePath();                         // o caminho completo
```

💡 **Caminho relativo** parte da pasta onde o programa roda; **absoluto** é o endereço completo. Use **relativo** nos projetos — assim funciona na máquina do colega também.

Escrevendo

```
Path arquivo = Path.of("dados", "notas.txt");
List<String> linhas = List.of("Ana;8.5", "Léo;6.0");
try {
    Files.createDirectories(arquivo.getParent());    // cria a pasta se faltar
    Files.write(arquivo, linhas);                    // SOBRESCREVE
} catch (IOException e) {
    System.out.println("Falha ao salvar: " + e.getMessage());
}
```

Para **acrescentar** em vez de sobrescrever:

```
Files.write(arquivo, linhas, StandardOpenOption.CREATE, StandardOpenOption.APPEND);
```

Dois avisos

`IOException` *é checked*: o compilador **obriga** a tratar com `try/catch` ou declarar `throws`.

`Files.write` **apaga o conteúdo anterior** por padrão.

Já perdi arquivo assim. Você também vai perder — uma vez.

Lendo

```
try {  
    if (!Files.exists(arquivo)) return;    // 1ª execução: normal, não é erro  
    for (String linha : Files.readAllLines(arquivo)) {  
        System.out.println(linha);  
    }  
} catch (IOException e) {  
    System.out.println("Falha ao ler: " + e.getMessage());  
}
```

Sempre verifique `Files.exists` antes de ler. Na primeira execução o arquivo ainda não existe — e isso é normal, não é erro.

De objeto para linha, e de volta

O arquivo guarda **texto**; o sistema trabalha com **objetos**.

```
// objeto → texto (na classe de modelo)
public String toCsv() {
    return String.join(";", codigo, titulo, String.valueOf(ano), autor);
}

// L001;Dom Casmurro;1899;Machado de Assis

// texto → objeto (static, porque ele CRIA o objeto)
public static Livro fromCsv(String linha) {
    String[] campos = linha.split(";");
    return new Livro(campos[0], campos[1], Integer.parseInt(campos[2]), campos[3]);
}
```

⚠ Escolha um separador que não apareça nos dados

Se um título tiver `;`, o seu `split` produz campos a mais.

Por isso `;` costuma ser melhor que `,` em texto em português.

(E por isso CSV de verdade tem regras de aspas.)

E trate a linha corrompida

`Integer.parseInt("mil oitocentos")` lança `NumberFormatException`.

Ao ler arquivo, **sempre** trate a linha ruim: pule com aviso, em vez de derrubar a carga inteira.

```
for (String linha : Files.readAllLines(ARQUIVO)) {  
    if (linha.isBlank()) continue;  
    try {  
        acervo.add(Livro.fromCsv(linha));  
    } catch (RuntimeException e) {  
        System.err.println("Linha ignorada: " + linha);  
    }  
}
```

Salvar e carregar o sistema inteiro

```
public void salvar() throws IOException {
    List<String> linhas = new ArrayList<>();
    for (ItemAcervo item : acervo) linhas.add(item.toCsv());
    Files.createDirectories(ARQUIVO.getParent());
    Files.write(ARQUIVO, linhas);
}

public void carregar() throws IOException {
    acervo.clear();
    if (!Files.exists(ARQUIVO)) return;    // arquivo inexistente = acervo vazio
    // ... o laço tolerante do slide anterior ...
}
```



Rode, cadastre, saia, abra de novo

O livro está lá.

Seu sistema tem memória.

Mas salvar só ao sair é frágil

Se o programa fechar de forma anormal, **tudo se perde**.

Opção mais segura: salvar após **cada** operação que altera dados.

 Em sistemas reais esse papel é do **banco de dados** — assunto do próximo curso da trilha.

Exercícios da aula

Na pasta `aula-13/` :

1. `Diario.java` — `APPEND` com data e hora; mostra as entradas ao iniciar;
2. `Contatos.java` — `toCsv` / `fromCsv` , carrega ao abrir, salva ao sair. **Rode duas vezes;**
3. `LeituraRobusta.java` — CSV com erros **de propósito**; carregue o que dá e não quebre;
4. `Relatorio.java` — leia vendas, calcule e **grave** o resultado num arquivo;
5. **Desafio** 🌶️ — persistência no projeto da aula 12: três arquivos, preservando os vínculos.

Próxima aula

Aula 14 — Lambdas e Streams

Boa parte dos laços que você escreveu
pode virar uma linha legível.